

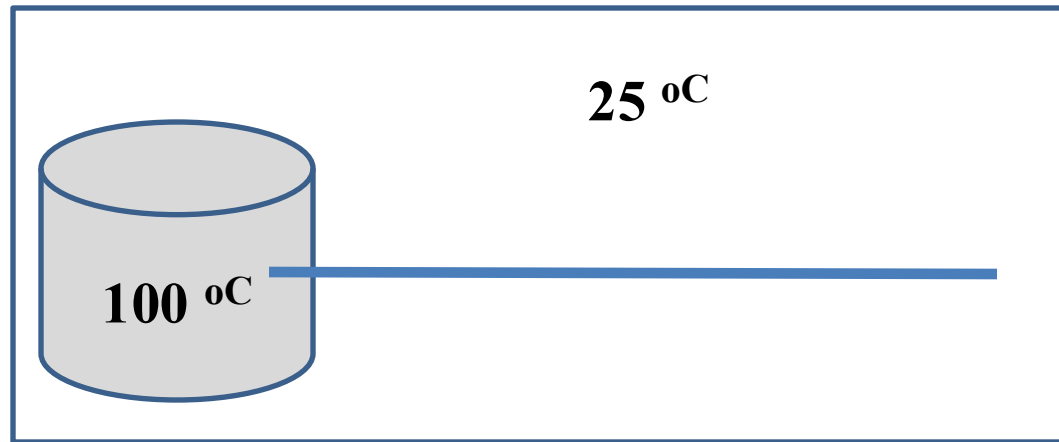
GIẢI THUẬT VÀ CÀI ĐẶT
CHƯƠNG TRÌNH SONG SONG

BÀI TOÁN CÓ SỰ PHỤ THUỘC DỮ LIỆU
PHƯƠNG TRÌNH NHIỆT 1D

Bài toán PTN 1D

- Bài toán PTN 1D:

Một thanh kim loại chiều dài L đặt trong phòng có nhiệt độ cố định 25°C . Một đầu của TKL cắm vào một nồi nước sôi 100°C . Nhiệt độ từ nồi nước sẽ khuếch tán dọc TKL. Hãy xác định nhiệt độ của TKL sau khoảng thời gian bằng Time?



Mô hình toán học

$$\frac{\partial T}{\partial t} = D \frac{\partial^2 T}{\partial x^2}$$

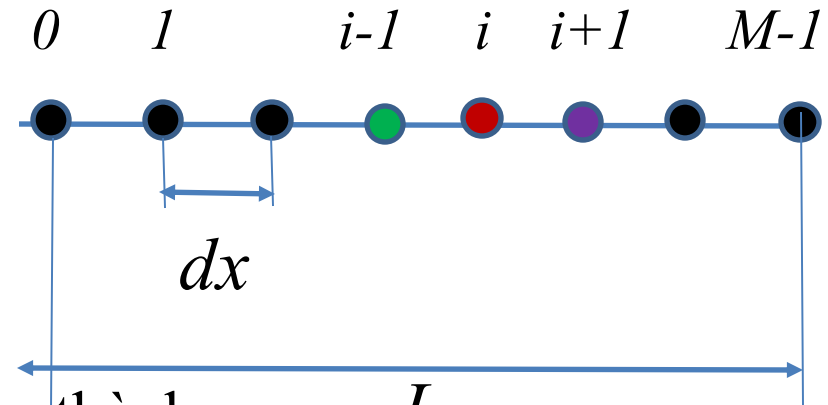
Trong đó:

- T là nhiệt độ
- t thời gian
- x là không gian 1 chiều
- D là hệ số truyền nhiệt

Phương pháp số giải PTN

- Rời rạc hóa:

- Theo không gian: Chia TKL chiều dài L thành $M-1$ đoạn độ dài dx , tạo thành một lưới M điểm, đánh chỉ số i chạy từ 0 đến $M-1$



- Theo thời gian: Chia thời gian Time thành các bước thời gian dt , đánh chỉ số là t chạy từ 0 đến Ntime

- CT sai phân thuận:

$$\frac{\partial T}{\partial t} = D \frac{\partial^2 T}{\partial x^2}$$

$$\frac{\partial^2 T_i}{\partial x^2} = \frac{(T_{i-1} - 2T_i + T_{i+1}))}{dx^2} \quad (1)$$

$$\frac{\partial T_i^t}{\partial t} = \frac{(T_i^{t+1} - T_i^t)}{dt} \quad (2)$$

Giải thuật

- Khởi tạo dữ liệu ban đầu:
 - Giả sử nhiệt độ của TKL bằng nhiệt độ phòng 25°C
$$T(i) = 25^{\circ}\text{C}, 0 < i < M-1$$
- Tại bước thứ $t+1$
 - Tính đạo hàm bậc hai của T theo x tại thời điểm t sử dụng công thức (1)
 - Tính giá trị của T tại thời điểm $t+1$ sử dụng công thức (2).
- Lưu ý: Điều kiện biên
 - Nhiệt độ tại điểm bên trái của điểm $i=0$ bằng 100°C
 - Nhiệt độ tại điểm bên phải của điểm $i=M-1$ bằng 25°C

Sự phụ thuộc dữ liệu

$$\frac{\partial^2 T_i}{\partial x^2} = \frac{(T_{i-1} - 2T_i + T_{i+1}))}{dx^2} \quad (1)$$

- Trong công thức (1) việc tính toán tại một điểm lưới (i) cần thông tin tại những điểm lưới xung quanh: (i-1), (i+1). Điều này gọi là sự phụ thuộc dữ liệu.
 - Đối với chương trình tuần tự: Cần điều kiện biên
 - Đối với chương trình song song (sử dụng mô hình bộ nhớ phân tán): Cần trao đổi thông tin (truyền thông) giữa các CPU, cần đồng bộ trong tính toán.

Cài đặt

- Ký hiệu:

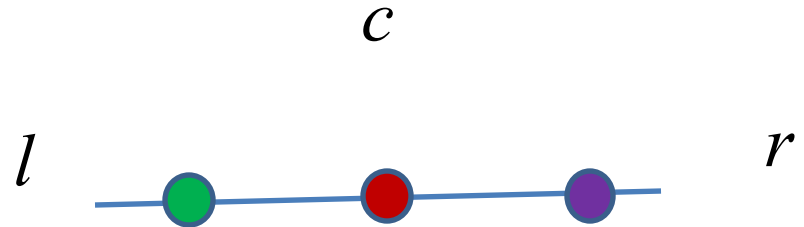
$$dT = D \frac{\partial^2 T}{\partial x^2},$$

$$c : T_i,$$

$$l : T_{i-1},$$

$$r : T_{i+1}.$$

$$\frac{\partial^2 T_i}{\partial x^2} = \frac{(T_{i-1} - 2T_i + T_{i+1}))}{dx^2} \quad (1)$$



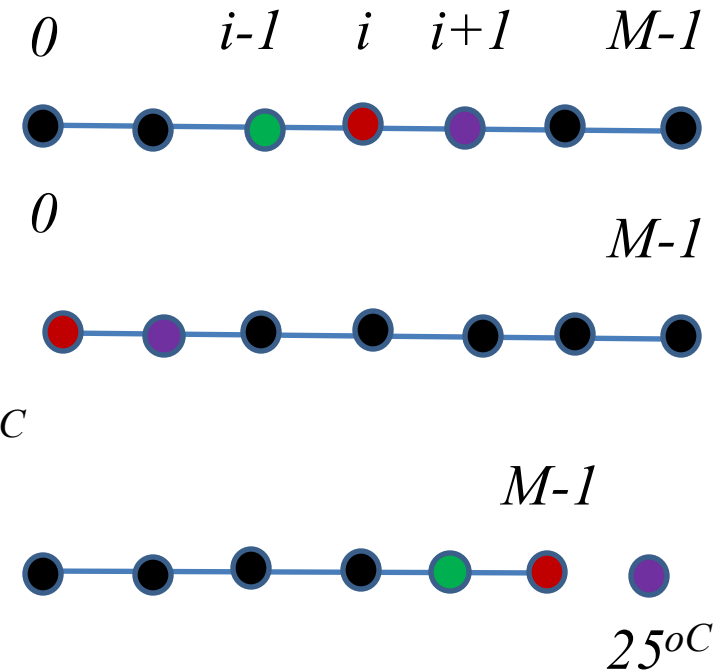
$$dT = D \frac{l - 2c + r}{dx^2} \quad (3)$$

Cài đặt hàm rời rạc theo không gian: DHB2

$$\frac{\partial^2 T_i}{\partial x^2} = \frac{(T_{i-1} - 2T_i + T_{i+1}))}{dx^2} \quad (1)$$

$$dT = D \frac{l - 2c + r}{dx^2} \quad (3)$$

```
void DHB2(float *T, float *dT) {
    int i;
    float c,l,r;
    for ( i = 0 ; i < M-1 ; i++ ) {
        c = *(T+i);
        l = (i==0)      ? 100 : *(T+i-1);
        r = (i==M-1)    ? 25  : *(T+i+1);
        *(dT+i) = D*(l-2*c+r)/(dx*dx);
    }
}
```



Cài đặt hàm tích hợp theo thời gian

$$D \frac{\partial^2 T}{\partial x^2} = dT = \frac{\partial T_i^t}{\partial t} = \frac{(T_i^{t+1} - T_i^t)}{dt}$$
$$\Rightarrow T_i^{t+1} = T_i^t + dT^t * dt$$

```
for (t = 0; t <= Ntime; t++)  
{  
    DHB2(T, dT);  
    for ( i = 0 ; i < M ; i++ )  
        *(T+i) = *(T+i) + *(dT+i)*dt;  
    t=t+dt;  
}
```

Bài tập: Viết chương trình C

- 1) Khởi tạo giá trị ban đầu
- 2) Tính toán giá trị T tại từng bước thời gian
 - Lưu mảng T tại từng bước thời gian ra file
- 3) Vẽ đồ thị mô phỏng sự thay đổi nhiệt độ của TKL
 - Có thể sử dụng Matlab

Giải thuật OpenMP

1) Khởi tạo

- ✓ Giống chương trình C

2) Chia miền tính toán

- ✓ Giả sử ta sử dụng NT threads, các luồng được đánh chỉ số (ID) từ 0 đến NT-1
- ✓ Chia miền tính toán gồm M điểm trong đoạn $[0, M-1]$ thành NT miền con, mỗi miền con được tính toán trên một thread, có kích thước M_c điểm, nằm trong đoạn $[start, stop]$, ta có:

$$M_c = M / NT$$

$$start = ID * M_c$$

$$stop = start + M_c$$

Giải thuật OpenMP

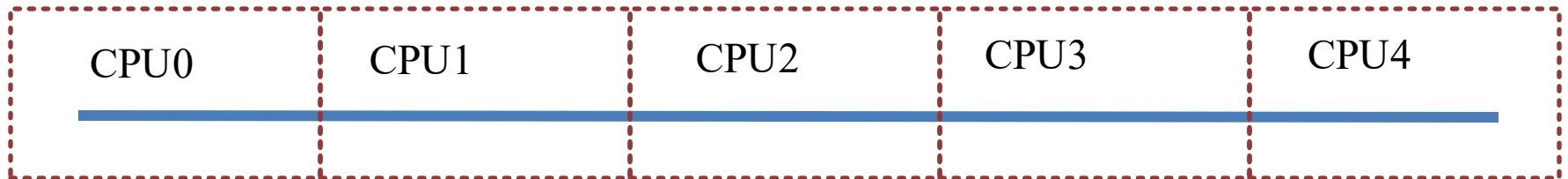
1) Tính toán song song trên NT threads

- ✓ Do có sự phụ thuộc dữ liệu trong tính toán nên cần phải đồng bộ
- ✓ Mỗi luồng chỉ tính toán trong miền con [start,stop]

2) Lưu kết quả ra file, mô phỏng

- ✓ Giống chương trình C

Giải thuật chương trình MPI



Domain
Decomposition

Giải thuật chương trình MPI

- Bước 1: Khởi tạo giá trị ban đầu tại CPU0 (root).
- Bước 2: Chia miền tính toán.
- Bước 3: Gửi Input từ root tới tất cả các CPU khác.
- Bước 4: Tính toán. Mỗi CPU tính toán trên subdomain tương ứng.
- Bước 5: Các CPU gửi kết quả tính toán về root.

Bước 3 và Bước 5: Truyền thông

Giải thuật chương trình MPI

- Bước 1: Khởi tạo giá trị ban đầu tại CPU0 (root)

Theo yêu cầu từng bài toán

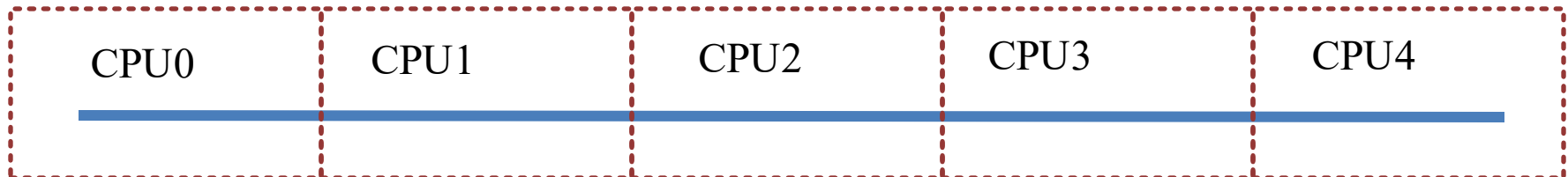
Ảnh hưởng đến kết quả đầu ra

Giải thuật chương trình MPI

- Bước 2: Chia miền tính toán.

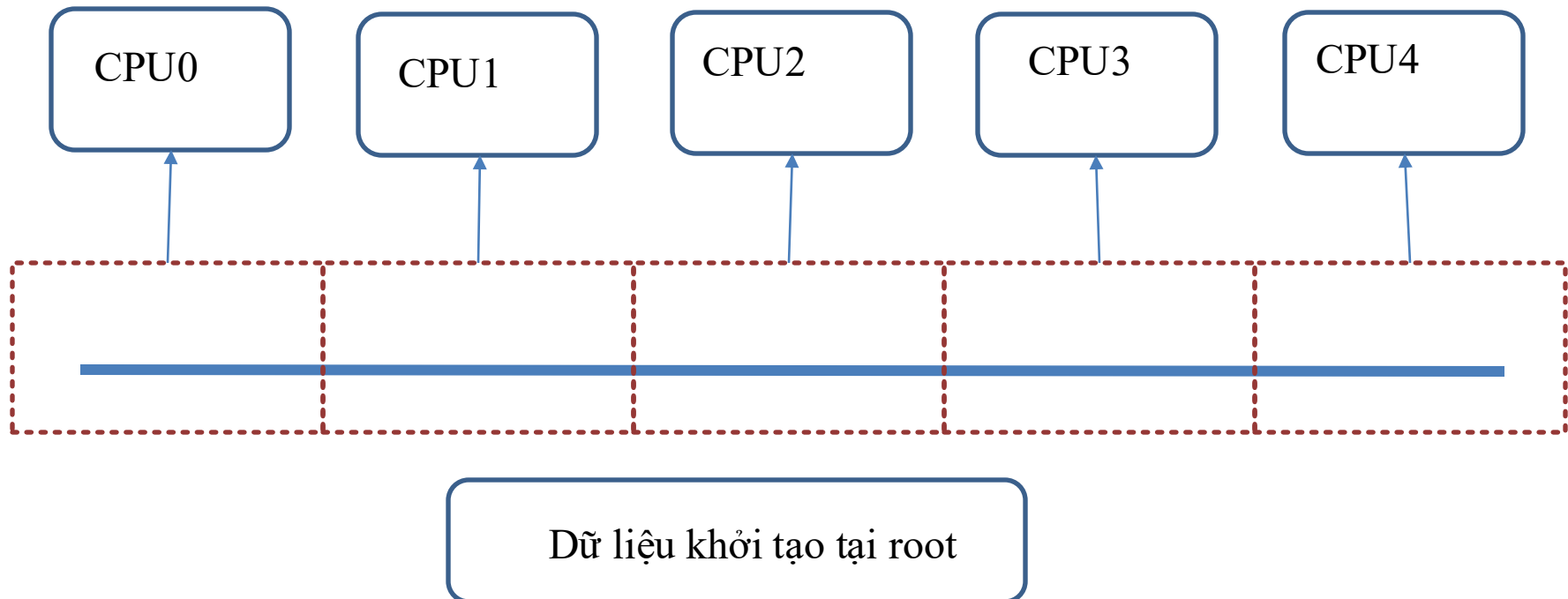
Kích thước miền tính toán là N .

Mỗi CPU tính toán trên miền có kích thước là $m_c = N / NP$ với NP là số lượng CPU



Giải thuật chương trình MPI

- Bước 3: Gửi Input từ root tới tất cả các CPU khác.
MPI_Scatter (T, mc, MPI_FLOAT,
Ts, mc, MPI_FLOAT,
0, MPI_COMM_WORLD);

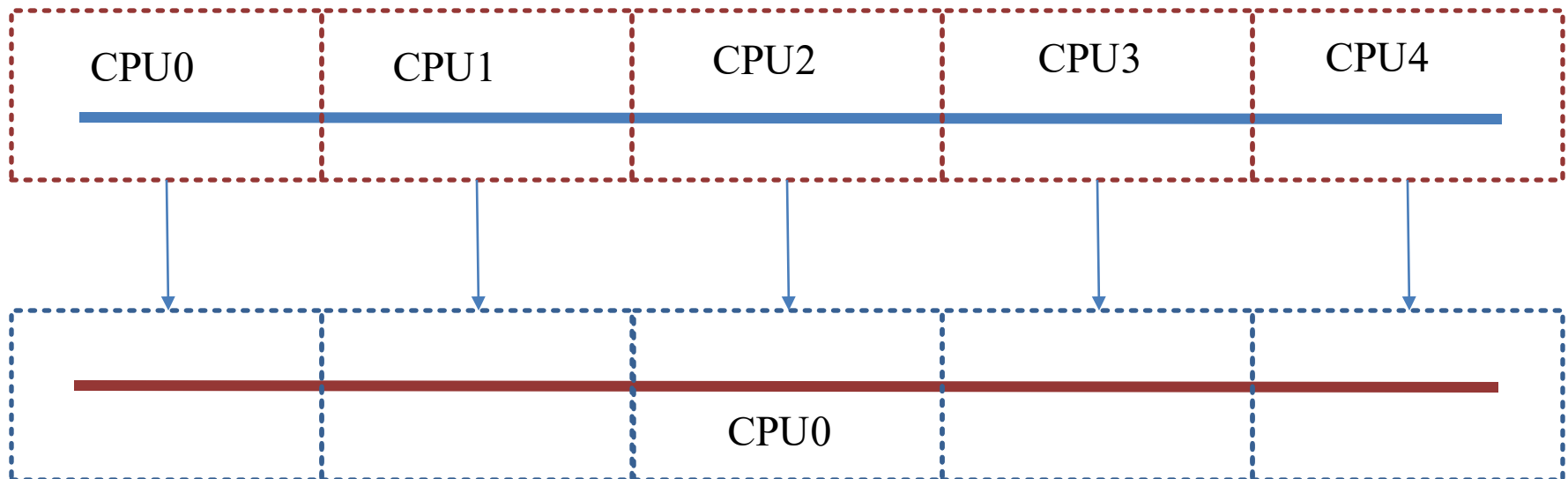


Giải thuật chương trình MPI

- Bước 5: Các CPU gửi kết quả tính toán về root.

`MPI_Gather (Ts, mc, MPI_FLOAT,
 T, mc, MPI_FLOAT, 0,
 MPI_COMM_WORLD);`

Kết quả tính toán tại các CPU

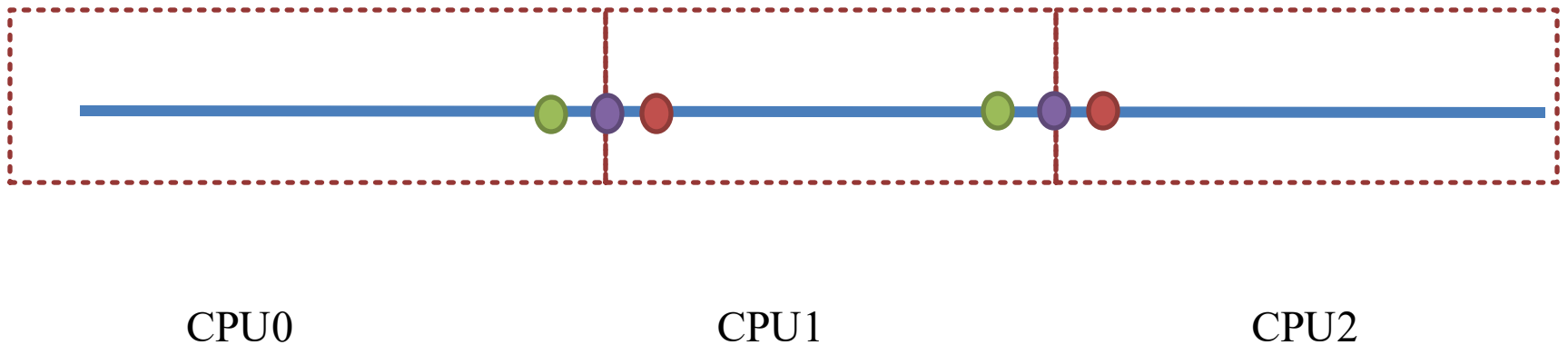


Giải thuật chương trình MPI

- Bước 4: Tính toán

B4.1 Truyền thông

B4.2 Tính toán

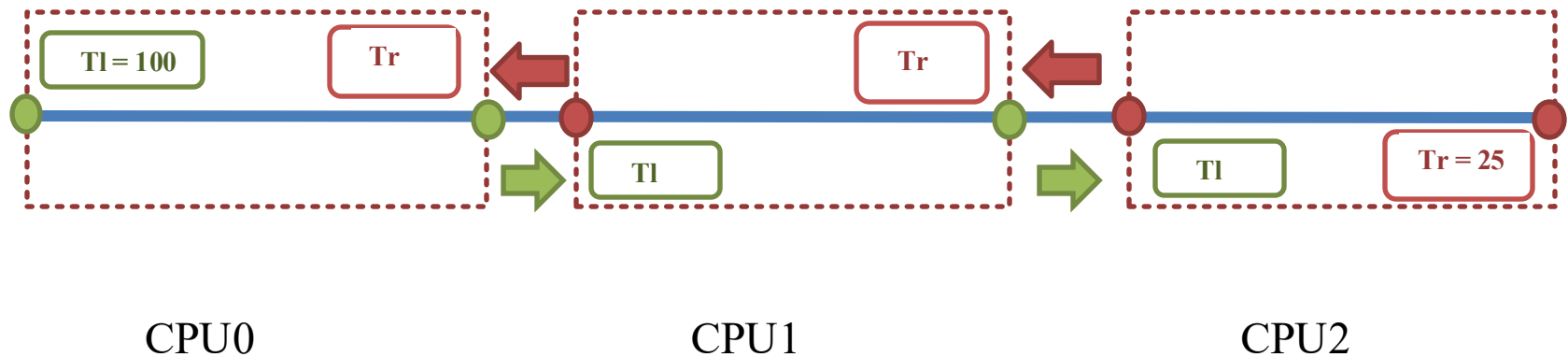


Giải thuật chương trình MPI

- Bước 4.1: Truyền thông

B4.1 - a Truyền thông phần tử Tl

B4.1 - b Truyền thông phần tử Tr



Giải thuật chương trình MPI

- Bước 4.1 - a: Truyền thông phần tử T1

```
if (rank == 0) {  
    T1 = 100;  
    MPI_Send (Ts + mc - 1, 1, MPI_FLOAT,  
              rank + 1, rank, MPI_COMM_WORLD);  
} else if (rank == size - 1) {  
    MPI_Recv (&T1, 1, MPI_FLOAT,  
             rank - 1, rank - 1, MPI_COMM_WORLD, &stat);  
} else {  
    MPI_Send (Ts + mc - 1, 1, MPI_FLOAT,  
              rank + 1, rank, MPI_COMM_WORLD);  
    MPI_Recv (&T1, 1, MPI_FLOAT,  
             rank - 1, rank - 1, MPI_COMM_WORLD, &stat);  
}
```

Giải thuật chương trình MPI

- Bước 4.1 - b: Truyền thông phần tử Tr

```
if (rank == size - 1) {  
    Tr = 25;  
    MPI_Send (Ts, 1, MPI_FLOAT,  
              rank - 1, rank, MPI_COMM_WORLD);  
} else if (rank == 0) {  
    MPI_Recv (&Tr, N, MPI_FLOAT,  
             rank + 1, rank + 1, MPI_COMM_WORLD, &stat);  
} else {  
    MPI_Send (Ts, 1, MPI_FLOAT,  
              rank - 1, rank, MPI_COMM_WORLD);  
    MPI_Recv (&Tr, 1, MPI_FLOAT,  
             rank + 1, rank + 1, MPI_COMM_WORLD, &stat);  
}
```

Giải thuật chương trình MPI

- Bước 4.2 - : Tính toán

```
void DHB2(float *Ts, float Tl, float Tr,
float *dT, int ms) {
    int i;
    float c, l, r;
    for (i = 0 ; i < ms ; i++) {
        c = *(Ts + i);
        l = (i == 0) ? Tl : *(Ts + i - 1);
        r = (i == ms - 1) ? Tr : *(Ts + i + 1);
        *(dT + i) = D*(l - 2 * c + r) / (dx * dx);
    }
}
```